

모두를 위한 딥러닝 (이미지처리편)

경북대학교 IT교육센터

허 찬

차시(모듈)	차시별 주제	학습내용
1	딥러닝을 이용한 이미지 처리	이미지 처리의 이해 다양한 이미지처리의 응용 분야에 대한 이해
2	특징 추출의 이해	이미지 데이터 전처리 기법의 이해 이미지 특징 추출의 이해 및 실습 딥러닝을 이용한 학습 및 추론 과정의 이해
3	Convolution Neural Network	합성곱 신경망의 이해
4	CNN 모델의 이해	다양한 CNN 모델의 이해 딥러닝의 여러가지 요소 이해
5	CNN을 이용한 분류 모델 실습	CNN을 이용한 이미지 분류 모델 실습
6	전이 학습 모델의 이해	전이 학습의 이해 전이학습을 이용한 이미지 처리 모델 실습
7	객체 인식 모델의 이해	다양한 객체 인식 모델의 이해 Semantic segmentation (U-Net)의 이해
8	Transformer 기반 이미지 처리 모델 1	Attention과 Transformer의 이해
9	Transformer 기반 이미지 처리 모델 2	Transformer 기반 이미지 처리 모델 (ViT)의 이해
10	이미지 생성 모델의 이해	최신 이미지 생성 모델의 이해 오토인코더의 이해 및 실습
11	최신 이미지 처리 기술의 이해	최신 컴퓨터비전 모델의 이해 비디오 처리의 이해

CNN을 이용한 분류 문제 예제

1. CNN기반 패션 분류 (이미지 데이터 - 흑백)
 - 흑백 이미지 분류모델 설계
2. CNN기반 cifar-10 이미지 분류 (이미지 데이터 - 컬러)
 - 컬러 이미지 분류모델 설계
3. Flower 데이터 카테고리 예측 (이미지 데이터 - 컬러)
 - 많은 클래스를 가지는 컬러 이미지 분류모델 설계

5.1 CNN을 이용한 Fashion 데이터 분류 모델

데이터셋 설명

- 목적: 10개의 클래스를 가지는 패션사진 정답 예측
- 5만개의 학습 데이터셋
- 1만개의 테스트 데이터셋

Input:

- 28x28 픽셀로 이루어진 흑백 이미지 데이터

Output:

- 최종적인 1개의 물체 class



Fashion mnist
dataset

5.1 CNN을 이용한 Fashion 데이터 분류 모델

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt

# 데이터셋 로드 및 전처리
fashion_mnist = tf.keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

train_images = train_images / 255.0
test_images = test_images / 255.0
```

5.1 CNN을 이용한 Fashion 데이터 분류 모델

```
# 클래스 이름 정의
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# 이미지 시각화 함수
def plot_images(images, labels, class_names, num_images=25):
    plt.figure(figsize=(10, 10))
    for i in range(num_images):
        plt.subplot(5, 5, i + 1)
        plt.xticks([])
        plt.yticks([])
        plt.grid(False)
        plt.imshow(images[i], cmap=plt.cm.binary)
        plt.xlabel(class_names[labels[i]])
    plt.show()

# 훈련 데이터셋의 25개 이미지 시각화
plot_images(train_images, train_labels, class_names)
```

5.1 CNN을 이용한 Fashion 데이터 분류 모델

```
model = Sequential()
model.add(Conv2D(32, kernel_size=(3,3), input_shape = (28,28,1), activation='relu'))
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=2))
model.add(Dropout(0.25))
model.add(Flatten())
model.add(Dense(10, activation='softmax')) ## 분류문제의 마지막 activation은 항상 softmax
##
model.summary()

# 모델 컴파일
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

# 모델 학습
model.fit(train_images, train_labels, epochs=10, validation_data=(test_images,
test_labels))
```

5.1 CNN을 이용한 Fashion 데이터 분류 모델

모델 평가

```
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
print('\n테스트 정확도:', test_acc)
```

예측 수행

```
probability_model = tf.keras.Sequential([model, tf.keras.layers.Softmax()])
predictions = probability_model.predict(test_images)
```

첫 번째 이미지 예측 결과 확인

```
print(predictions[0])
print("예측된 레이블:", predictions[0].argmax())
print("실제 레이블:", test_labels[0])
```

5.2 CNN을 이용한 Cifar-10 데이터 분류 모델

데이터셋 설명

- 목적: 10개의 클래스를 가지는 동물사진 정답 예측
- 5만개의 학습 데이터셋
- 1만개의 테스트 데이터셋

Input:

- 28x28 픽셀로 이루어진 컬러 이미지 데이터

Output:

- 최종적인 1개의 물체 class

비행기

자동차

새

고양이

사슴

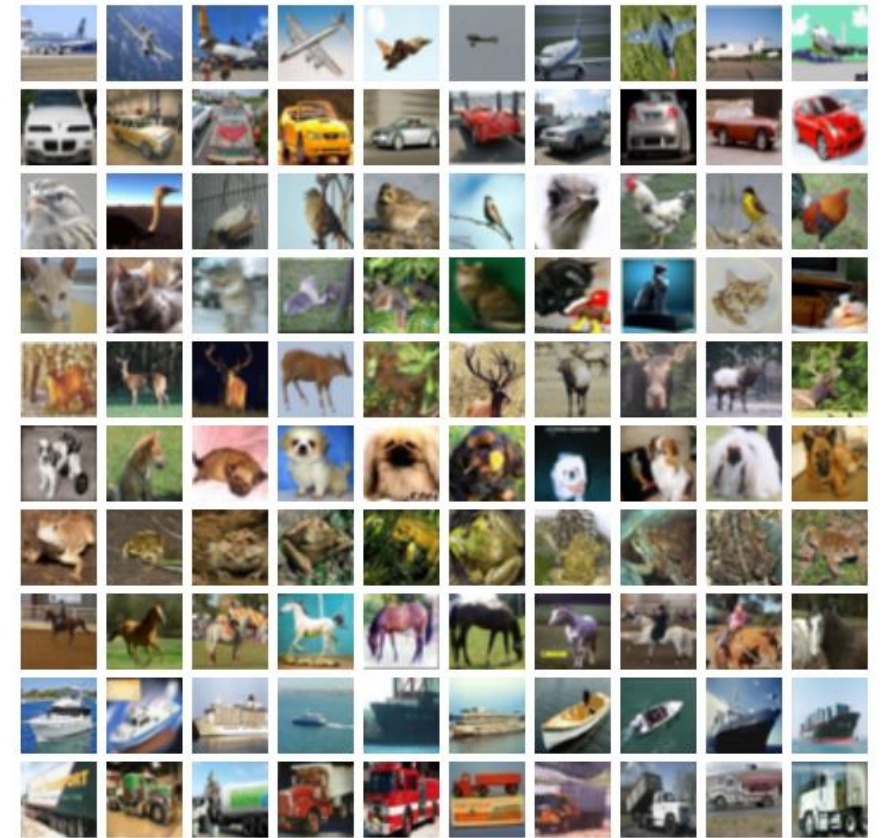
개

개구리

말

배

트럭



Cifar-10 dataset

5.2 CNN을 이용한 Cifar-10 데이터 분류 모델

1) Module import

```
# 사용할 라이브러리 불러오기
import numpy as np
from keras.datasets import cifar10
from keras.models import Sequential
from keras.layers import Dense, Activation, Conv2D, MaxPooling2D, Flatten
from keras.utils import to_categorical
import matplotlib.pyplot as plt
```

5.2 CNN을 이용한 Cifar-10 데이터 분류 모델

2) data load 및 시각화

```
# CIFAR-10 Dataset 가져오기
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
print('x_train :', np.shape(x_train))
print('y_train :', np.shape(y_train))
print('x_test :', np.shape(x_test))
print('y_test :', np.shape(y_test))

# 이미지 예제 시각화
plt.subplot(121)
plt.imshow(x_train[0], interpolation="bicubic")
plt.grid(False)
plt.subplot(122)
plt.imshow(x_train[2], interpolation="bicubic")
plt.grid(False)
plt.show()
```

5.2 CNN을 이용한 Cifar-10 데이터 분류 모델

3) data 전처리

```
# 데이터 스케일링
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# 라벨 One-Hot Encoding
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
print('y_train :', np.shape(y_train))
```

5.2 CNN을 이용한 Cifar-10 데이터 분류 모델

4) Model build

```
# 모델 구성하기
width = 32
height = 32
channel = 3
# 순차적 모델을 만들기 위한 Sequential() 함수
model = Sequential()
# filters : Kernel 그룹의 갯수 / kernel_size : Kernel 크기
model.add(Conv2D(filters=32, kernel_size=(3, 3), padding='same', activation='relu',
                 input_shape=(width, height, channel)))
# pool_size : MaxPooling 의 Window size
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(filters=64, kernel_size=(3, 3), padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(filters=128, kernel_size=(3, 3), padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
# N차원 배열을 1차원으로 펴줍니다
model.add(Flatten())
# 출력층 MLP
model.add(Dense(10, activation='softmax'))
model.summary()
```

- 문제에 따른 Loss, activation 항상 주의

5.2 CNN을 이용한 Cifar-10 데이터 분류 모델

5) Compile & training

모델 구성 및 학습

```
model.compile(loss='categorical_crossentropy', optimizer='sgd', metrics=['accuracy'])  
history = model.fit(x_train, y_train, epochs=20, batch_size=64, validation_data=(x_test, y_test))
```

##history는 결과를 저장하는 객체##

5.2 CNN을 이용한 Cifar-10 데이터 분류 모델

6) 모델 평가

```
#모델 평가
loss_and_acc = model.evaluate(x_test, y_test, batch_size=32)
print('Test set Loss and Accuracy')
print(loss_and_acc)
```

5.3 CNN을 이용한 Flower 데이터 분류 모델

데이터셋 설명

- 목적: 102개의 클래스를 가지는 꽃사진 정답 예측

Input:

- 150x150x3 픽셀로 이루어진 컬러 이미지 데이터

Output:

- 102개의 물체 class 중 1개 정답



water lily (72)



desert-rose (84)



gazania (70)



wild pansy (51)



oxeye daisy (48)



columbine (83)



sword lily (42)



orange dahlia (58)



barbeton daisy (40)

5.3 CNN을 이용한 Flower 데이터 분류 모델

```
import tensorflow as tf
import tensorflow_datasets as tfds

# 데이터셋 로드
(dataset_train, dataset_test), dataset_info = tfds.load(
    'oxford_flowers102',
    split=['train', 'test'],
    with_info=True,
    as_supervised=True
)

# 데이터 전처리 함수
def format_example(image, label):
    image = tf.image.resize(image, (150, 150))
    image = image / 255.0
    return image, label

# 데이터셋 전처리
train_batches = dataset_train.map(format_example).batch(32).prefetch(1)
test_batches = dataset_test.map(format_example).batch(32).prefetch(1)
```

5.3 CNN을 이용한 Flower 데이터 분류 모델

```
def display_images(dataset, num_images):  
    plt.figure(figsize=(10, 10))  
    for images, labels in dataset.take(1):  
        for i in range(num_images):  
            ax = plt.subplot(5, 5, i + 1)  
            plt.imshow(images[i])  
            plt.title(int(labels[i]))  
            plt.axis("off")  
  
# 훈련 데이터셋에서 25개의 이미지 시각화  
display_images(train_batches, 25)
```

5.3 CNN을 이용한 Flower 데이터 분류 모델

```
from tensorflow.keras import layers, models

# 모델 구성
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)),
    layers.MaxPooling2D(2, 2),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Flatten(),
    layers.Dropout(0.5),
    layers.Dense(512, activation='relu'),
    layers.Dense(102, activation='softmax')
])

# 모델 컴파일
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

5.3 CNN을 이용한 Flower 데이터 분류 모델

```
# 모델 학습
history = model.fit(
    train_batches,
    epochs=3,
    validation_data=test_batches
)

# 모델 평가
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(len(acc))

import matplotlib.pyplot as plt

plt.plot(epochs, acc, 'r', label='Training accuracy')
plt.plot(epochs, val_acc, 'b', label='Validation accuracy')
plt.title('Training and validation accuracy')
plt.legend()
plt.show()

plt.plot(epochs, loss, 'r', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.legend()
plt.show()
```

감사합니다